

Towards Efficient Code Generation with LLMs: Next Generation Metrics and Preference Optimization

Hanbeul Kim Hyunsu Moh Junhyeog Yun Juhee Han Jaehwan Lee
Seoul National University

{kimhan0515, hyunsumoh, junhyeog, onerat}@snu.ac.kr jaehwan@aces.snu.ac.kr

Abstract

The advent of specialized Large Language Models (LLMs) for code generation, or code LLMs, has transformed the automation of programming tasks by enabling the generation of code solutions from natural language problem descriptions. While benchmarks like HumanEval and MBPP evaluate functional correctness, they often overlook code efficiency, which is a critical aspect of code quality. To bridge this gap, efficiency-aware benchmarks such as Mercury and EffiBench have been introduced, along with fine-tuning strategies to enhance efficiency while preserving correctness. However, existing efficiency-aware metrics often rely on over-simplified assumptions about runtime distributions, limiting their robustness. Moreover, existing fine-tuning methods, such as Direct Preference Optimization (DPO), while effective, have yet to incorporate advancements from newer algorithms like IPO and CPO. In this work, we propose BeyondX, a novel efficiency metric that uses the cumulative distribution function (CDF) of runtime distributions for more robust evaluations. We also construct a preference dataset to guide LLMs toward generating efficient yet correct code and demonstrate experimentally that combining DPO with IPO further enhances LLMs to generate more efficient code. Our findings contribute to advancing the evaluation and optimization of code LLMs, providing a robust framework for generating efficient and reliable code.

1. Introduction

Recently, Large Language Models (LLMs) have advanced rapidly, enabling the automation of various natural language processing (NLP) tasks. Leveraging their ability to understand problem descriptions written in natural language and generate corresponding solutions in code, specialized LLMs for code generation, code LLMs, have emerged.

Consequently, to evaluate and compare code LLMs, numerous benchmarks have been proposed to evaluate the

quality of the generated code. Early benchmarks, such as HumanEval [4] and MBPP [1], are primarily focused on evaluating functional correctness. However, they often overlook efficiency, which is also a critical aspect of code quality. To address this limitation, recently, efficiency-aware benchmarks such as Mercury [5] and EffiBench [7] were devised. Furthermore, [5] provided fine-tuning strategies for code LLMs to effectively enhance efficiency while preserving functional correctness.

Despite these advancements, there still remains room for improvement. While existing efficiency-aware metrics normalize the estimated code efficiency based on the efficiency distribution on possible solutions of a given problem, they are often based on an overly simplistic assumption for the efficiency distribution. For instance, the efficiency-aware metric from Mercury assumes a uniform distribution of code efficiency for simplicity. However, this simplicity cannot reflect the true distribution, reducing robustness to variation in distribution. In addition, further exploration is needed for the fine-tuning algorithm. [5] compared supervised fine-tuning (SFT) [3] and direct preference optimization (DPO) [9] method and concluded that preference optimization algorithms offer more stable enhancement in code efficiency than simple SFT. However, DPO may also not be an optimal strategy, as improved algorithms such as Identity Preference Optimization (IPO) [2], Contrastive Preference Optimization (CPO) [10], and Odds Ratio Preference Optimization (ORPO) [6] were later proposed. Analyzing and comparing these new algorithms may lead to further enhancements in code efficiency. In this work, we aim to improve the existing efficiency-aware benchmark, Mercury, in terms of metric and preference optimization.

Our key contributions are as follows:

- We propose a novel code efficiency metric, BeyondX, which leverages the cumulative distribution function (CDF) of runtime distribution to enhance robustness, while addressing the limitation of previous approaches.
- We construct a preference dataset that helps LLMs generate more efficient code while preserving correctness.
- We demonstrate that combining DPO with IPO improves

the ability of LLMs to generate efficient code.

2. Background and Related Work

This section provides an overview of evaluation metrics and fine-tuning methods that contribute to efficient code generation with LLMs.

2.1. Correctness-oriented Benchmarks

Correctness remains a primary focus in evaluating the quality of LLM-generated code, with benchmarks like HumanEval [4] and Mostly Basic Python Problems (MBPP) [1] playing a significant role. HumanEval consists of a curated set of Python programming problems designed to assess functional correctness, specifically measuring the model’s ability to generate code that passes predefined unit tests. Similarly, MBPP offers a broader collection of Python tasks, covering a range of basic programming constructs and scenarios to evaluate correctness across diverse problem types. While these benchmarks provide valuable insights into code generation accuracy and functional correctness, they fail to computational efficiency, an equally important dimension for real-world applications. This omission leaves a significant gap in holistic performance evaluation. For instance, two solutions that are functionally correct may differ drastically in runtime or resource usage, yet such distinctions are overlooked in correctness-only assessments.

2.2. Efficiency-aware Benchmarks

EffiBench [7] and Mercury [5] propose benchmarks to evaluate the efficiency of code generated by code generation models. Mercury [5] is a benchmark designed to evaluate the efficiency of code generated by LLMs. It includes 1,889 programming problems collected from LeetCode [8]. To measure code efficiency, the authors introduce a metric called *Beyond*, which compares the runtime of LLM-generated code to the runtime distribution of historical solutions for each problem. The runtime of each code is measured using test cases. GPT-4 is first used to generate a test case generator code, which is then applied to create the test cases needed to evaluate the code’s correctness and runtime. However, this method has limitations. It is heavily influenced by outliers, such as solutions with very high time complexity, and it assumes that the runtime distribution of historical solutions is linear. These factors make it difficult to assess the efficiency of the generated code accurately. EffiBench [7] consists of 1,000 programming problems, each paired with a canonical solution, which is the best known human-written solution from LeetCode [8]. To assess solutions for these problems, EffiBench uses test cases generated by GPT-3.5-turbo. Specifically, GPT-3.5-turbo first produces test case generation code, which is then used to create 100 test cases for each problem. EffiBench measures

the efficiency of solutions using six straightforward metrics: execution time, peak memory usage, and total memory usage, along with their normalized versions: Normalized Execution Time (NET), Normalized Memory Usage (NMU), and Normalized Total Memory Usage (NTMU), calculated using the canonical solution as a baseline. EffiBench uses a variety of metrics that heavily depend on the execution environment, making it difficult to robustly measure code efficiency.

2.3. Preference Optimization

Preference optimization aims to fine-tune LLMs by aligning their outputs with user preferences or predefined objectives. These methods address the limitations of traditional likelihood-based training, focusing on improving specific metrics, such as efficiency, correctness, or user satisfaction. Direct Preference Optimization (DPO) [9] directly optimizes the log-likelihood ratio between preferred and less-preferred outputs, removing the need for an intermediate reward process. Identity Preference Optimization (IPO) [2] extends DPO by introducing a regularization term to control the gap between likelihood ratios, mitigating overfitting issues. Contrastive Preference Optimization (CPO) [10] leverages contrastive learning to maximize the distinction between preferred and less-preferred outputs, while Odds Ratio Preference Optimization (ORPO) [6] focuses on computational efficiency and training stability, making it suitable for large-scale preference optimization tasks. These methods play a crucial role in enhancing fine-tuned models, particularly for tasks requiring preference alignment, such as code generation.

3. Code Efficiency Metric

3.1. The Proposed Metric: BeyondX

The metric used in the previous work, *Beyond* [5], quantifies the efficiency of code by calculating the relative percentile of the LLM-generated solution among the historical human solutions for a given task. For a dataset consisting of N tasks, the LLM under evaluation generates K sample solutions for each task, and the percentile score p_k^n for the k -th sample solution of the n -th task is calculated based on its measured runtime r_k^n and the runtime array R^n of the historical solutions for the n -th task. The *Beyond* score is calculated as the average of p_k^n across all sample solutions for all tasks as follows:

$$p_k^n = \frac{\max(R^n) - \text{clip}(r_k^n, \min(R^n), \max(R^n))}{\max(R^n) - \min(R^n)} \quad (1)$$

$$\text{Beyond} = \frac{\sum_{n=0, k=0}^{N, K} p_k^n}{N \cdot K} \quad (2)$$

However, a key limitation of the `Beyond` metric is its assumption that the runtime distribution of historical solutions is uniform, which may not hold true in real-world scenarios. While this assumption simplifies the evaluation process, it limits the reflection of the actual runtime distribution, leading to inaccurate evaluation for most samples, and making it vulnerable to variances in the distribution.

To address the limitations of previous work, we propose a new metric, `BeyondX`, as an improved version of `Beyond`. To better capture the full scope of the runtime distribution and improve evaluation accuracy, `BeyondX` utilizes the cumulative distribution function (CDF) of the actual runtime distribution, approximated using linear interpolation. As shown in Equation 3, the CDF is interpolated using a piecewise linear method.

$$CDF(r_k^n) = \begin{cases} 0 & \text{if } r_k^n \leq \min(R^n), \\ 1 & \text{if } r_k^n > \max(R^n), \\ i + \frac{r_k^n - R^n[i]}{R^n[i+1] - R^n[i]} & \text{else,} \end{cases} \quad (3)$$

$$p_k^n = 1 - CDF(r_k^n) \quad (4)$$

$$\text{BeyondX} = \frac{\sum_{n=0, k=0}^{N, K} p_k^n}{N \cdot K} \quad (5)$$

3.2. Further Improvements: Clustering

While `BeyondX` provides exact percentile to evaluate the efficiency by capturing the entire runtime distribution of historical solutions, there is still a potential issue with calculating the CDF using linear interpolation. The CDF value changes abruptly in regions with densely concentrated historical runtime values, causing small fluctuations in the runtime of generated code to result in significant changes in the metric score.

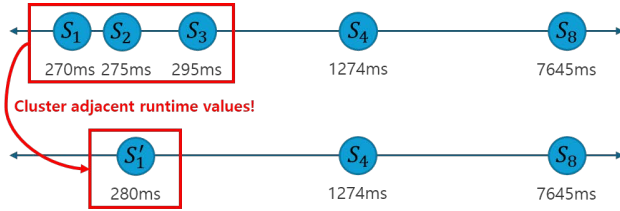


Figure 1. **Intuitive example of clustering.** In this example, historical solutions of S_1 (270 ms), S_2 (275 ms), and S_3 (295 ms) are clustered, and only the averaged runtime value (280 ms) is considered when evaluating code efficiency.

To deal with this problem, a straightforward approach is to cluster solutions that have similar runtimes as illustrated in Figure 1. When evaluating `BeyondX`, runtimes of the historical solutions whose differences are within a certain

threshold are clustered and the mean runtime of the clustered solutions is used.

4. Preference Optimization

The Mercury benchmark [5] comprises 1,633 coding problems sourced from LeetCode [8]. Each problem in the Mercury dataset is associated with an average of 14 historical solutions, and each solution’s runtime is evaluated in a sandbox environment. For preference optimization methods like DPO, we need a preference dataset which consists of (*chosen*, *rejected*) pairs for given problems. For each problem as a prompt, efficient code is selected as the chosen completion, while inefficient code is assigned as the rejected completion. For a simplified example, consider the prompt: “Write a code that sorts a list of numbers.” A plausible example of a chosen completion would be a quick-sort implementation with $O(n \log n)$ complexity, while a rejected completion would be a bubble sort implementation with $O(n^2)$ complexity. The authors of Mercury [5] create preference pairs by selecting combinations of two different historical solutions for each problem, where the difference in runtime is at least 20 ms. They utilize the constructed preference dataset to fine-tune LLMs using Supervised Fine-Tuning (SFT) [3] or Direct Preference Optimization (DPO) [9], aiming to optimize LLMs to generate not only functionally correct but also computationally efficient solution code. Despite its contribution, we identify several limitations in Mercury’s approach to preference optimization. The fixed threshold of 20 ms for runtime differences may fail to capture nuanced efficiency distinctions, particularly for problems with low runtime variability. Additionally, the dataset does not account for cases where historical solutions exhibit similar runtime but differ significantly in implementation quality or resource utilization, limiting its ability to guide LLMs toward generating truly efficient code. Furthermore, while fine-tuning methods like DPO show promise, they remain underexplored, especially in terms of integrating newer and potentially more effective algorithms for preference optimization.

4.1. Dynamic Runtime Threshold

Figure 2 shows the mean value (red line) and box plots of the historical solutions’ runtimes for all coding problems in Mercury’s train dataset presented on a log-scale. The runtime distributions of historical solutions across problems vary significantly in both their mean and standard deviation. This variability makes it problematic to use a static threshold, such as the 20 ms difference, to distinguish between chosen and rejected completions. A static threshold fails to reflect the unique characteristics of each problem’s runtime distribution. To address this issue, we apply a dynamic threshold value. Specifically, we define the threshold as a value proportional to the standard deviation of the runtime

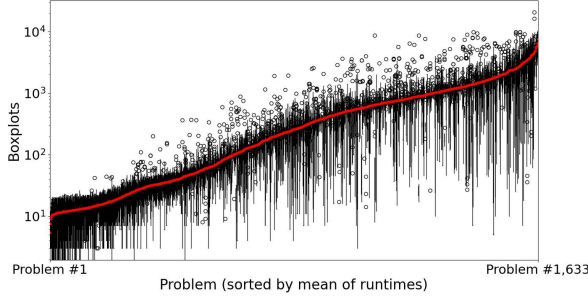


Figure 2. **Runtime distributions for each problem’s historical solutions in Mercury’s train dataset.** Presented on a log scale, the red line indicates the mean runtime, while the dots and lines represent runtime variability.

distribution for each problem, as follows:

$$Threshold_i = 1.8 \times \sigma_i, \quad (6)$$

where $Threshold_i$ is the threshold for problem i and σ_i is the standard deviation of the runtime distribution of problem i . This approach ensures that the preference dataset more accurately captures the characteristics of each coding problem, improving the overall quality of preference optimization.

4.2. Balanced Preference Pairs

The Mercury’s train dataset exhibits significant variation in the number of historical solutions per problem, ranging from as few as 2 to as many as 121. Consequently, the number of preference pairs per problem can range from 1 to 7,260. This imbalance introduces a bias in the preference dataset, as problems with more historical solutions dominate the dataset. For instance, a problem with 121 historical solutions generates 7,260 preference pairs, whereas a problem with only two historical solutions generates just one pair. To mitigate this imbalance, we limit the number of preference pairs per problem to a maximum of 5. By doing so, we ensure that the preference dataset remains balanced and prevents any single problem from disproportionately influencing the preference optimization process. This method helps stabilize preference optimization and enhances its overall reliability.

4.3. Combining DPO with IPO

We leverage the preference dataset to optimize LLMs to generate functionally correct and computationally efficient code. The authors of Mercury fine-tuned LLMs using SFT (Supervised Fine-Tuning) [3] and DPO (Direct Preference Optimization) [9] methods with the preference dataset. While SFT is a straightforward approach, it carries the risk of catastrophic forgetting, where the model loses its general performance while aligning to specific preferences. To

address this, recent research has focused on preference optimization methods such as DPO, CPO (Contrastive Preference Optimization) [10], and ORPO (Odds Ratio Preference Optimization) [6], which aim to fine-tune LLMs to align with preferences while preserving their general performance. Building on this work, we further enhance DPO and CPO by incorporating the IPO (Identity Preference Optimization) [2] loss function, which prevents overfitting by regularization to improve performance stability.

5. Evaluation

In this section, we conduct a comprehensive evaluation of whether *BeyondX* can serve as a reliable code efficiency metric, and demonstrate how the proposed approach to build the preference dataset enhances the ability of LLMs to generate more efficient code.

5.1. Experimental Setup

System Configuration. The experiments are conducted on a system equipped with a single AMD EPYC 7452 32-core CPU, four NVIDIA GeForce RTX 2080 GPUs, and a total memory capacity of 256 GiB.

Models and Datasets. We evaluate the proposed approaches on two models, DeepSeek Coder 1.3B and DeepSeek Coder 6.7B. Preference optimization is performed using the training dataset from Mercury, while the evaluation dataset is used to compute the metric scores.

5.2. Robustness of Metrics

The primary goal of a metric is to deliver accurate and reliable evaluations. We evaluate the robustness and reliability of the metrics by introducing slight modifications to the runtime distributions of solutions in the dataset. As shown in Table 1, *Beyond* shows unstable results with subtle changes in the runtime distribution when removing the min/max runtime solutions or randomly removing two solutions, as *Beyond* measures the efficiency of the code with only the minimum and maximum runtime values. However, since *BeyondX* measures the efficiency using entire solutions, the score remains nearly invariant despite changes in distribution.

Metric	Original	Min/Max Removal	Random Removal
<i>Beyond</i>	34.14	36.38 (+2.24)	36.31 (+2.17)
<i>BeyondX</i>	27.26	27.48 (+0.22)	27.19 (-0.07)

Table 1. **Comparison of sensitivity to distribution modification.** ‘Original’ refers to the complete runtime distribution, ‘Min/Max Removal’ excludes the fastest and slowest solutions, and ‘Random Removal’ eliminates two arbitrary solutions. Values in parentheses show changes relative to the ‘Original’.

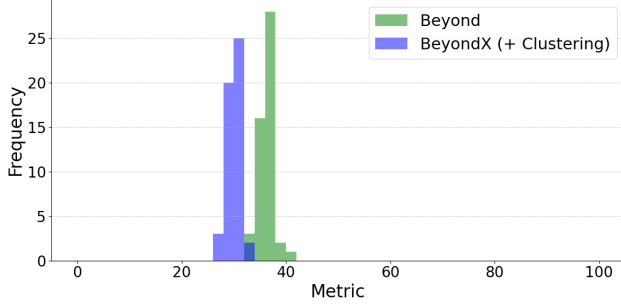


Figure 3. **Comparison of robustness to repeated measurements.** We conduct 50 trials to illustrate how slight runtime fluctuations can lead to variations in the code efficiency metrics.

An efficiency-aware metric can naturally fluctuate due to runtime errors and variations. To assess robustness under runtime fluctuations, we compare *Beyond* and *BeyondX* by repeatedly measuring their metric scores. The results indicate that both metrics produce similar scores, but *BeyondX* achieves higher stability in its metric scores under varying runtime conditions. As shown in Figure 3, *Beyond* exhibits a variation gap of approximately 6, whereas *BeyondX* reduces this gap to approximately 4. This suggests that *BeyondX* effectively reduces the impact of minor runtime fluctuations, resulting in more stable and reliable evaluations.

Metric	Avg. rate of metric changes for a <u>0.1 ms</u> change in runtime ($\downarrow$)
W/o Clustering	26.08%
W/ Clustering	16.26%

Table 2. **Comparison of robustness with and without clustering.** A lower percentage indicates that the measured metric is less sensitive to small changes in runtime, suggesting improved stability.

5.3. Effectiveness of Clustering

We further develop *BeyondX* by clustering solutions with similar runtime values to further enhance robustness, as discussed in Section 3.2. To assess the effectiveness of the clustering approach, we introduce an artificial 0.1 ms fluctuation to the runtime of generated code, simulating the inevitable variations that occur during actual runtime measurements. The results in Table 2 demonstrate that the clustering approach substantially improves the robustness of *BeyondX*, reducing the average rate of metric changes due to a 0.1 ms fluctuation in runtime from 26.08% to 16.26%.

5.4. Computational Overhead

Beyond estimates the relative percentile using only the minimum and maximum values of the runtime distribution.

Metric	Avg. overhead per sample ($\downarrow$)	Total overhead ($\downarrow$)
Beyond	0.0025 ms	0.6475 ms
BeyondX	0.0070 ms	1.7979 ms

Table 3. **Comparison of metric computational overhead.** The total overhead represents the cumulative time added across all samples (or tasks) in the evaluation dataset.

In contrast, *BeyondX* calculates the exact percentile by leveraging the CDF, which incurs slightly higher computational overhead. However, as shown in Table 3, the total overhead for metric calculation across all samples in the evaluation dataset remains under 2 ms, making it negligible.

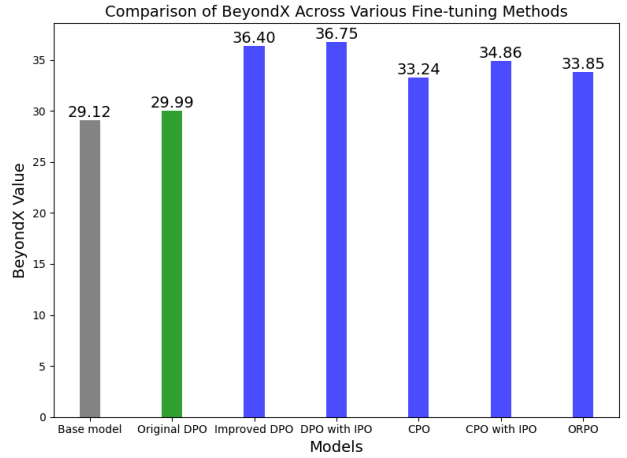


Figure 4. **Comparison of *BeyondX* for models fine-tuned with various preference optimization methods.** The base model is DeepSeek Coder 1.3B.

5.5. Preference Optimization Fine-tuning

Figure 4 presents the *BeyondX* scores of fine-tuned LLMs using various preference optimization methods. Our observations are as follows:

- Using the original preference dataset from the Mercury authors, DPO fine-tuning yields only a slight improvement (29.99 compared to 29.12).
- In contrast, applying DPO with the proposed preference dataset, described in Sections 4.1 and 4.2 results in an evident increase (36.40 compared to 29.12).
- With the improved dataset, DPO, CPO, and ORPO achieve *BeyondX* scores of 36.40, 33.24, and 33.85, respectively. While all methods enhance code efficiency, DPO attains the highest performance.
- Incorporating IPO into DPO and CPO boosts their scores even further, especially for CPO (36.75 compared to 36.40 for DPO, and 34.86 compared to 33.24 for CPO).

To check generality and applicability to larger models,

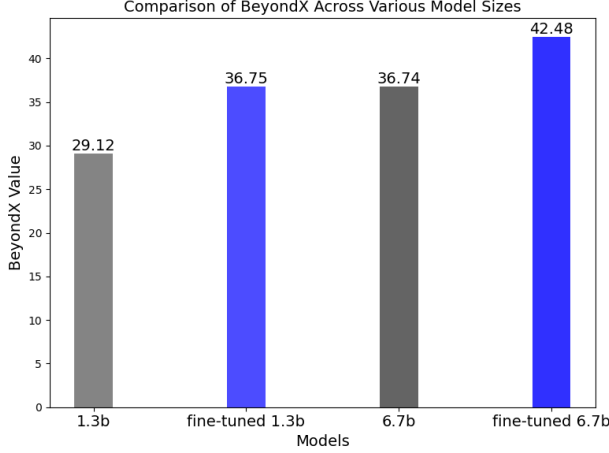


Figure 5. **Comparison of BeyondX improvements for DeepSeek Coder 1.3B and 6.7B models fine-tuned using DPO and IPO with the improved preference dataset.**

we apply the preference optimization fine-tuning to a bigger model, DeepSeek Coder 6.7B. Figure 5 shows that preference-based fine-tuning improves the BeyondX score from 36.74 to 42.48 in the 6.7B model. This consistent performance improvement in various model sizes demonstrates that preference optimization fine-tuning is also capable of enhancing LLMs to generate more efficient codes in terms of the proposed metric, BeyondX.

<pre> 1 def merge(self, nums1, m, nums2, n): 2 nums1_copy = nums1[:m] 3 p1 = 0 4 p2 = 0 5 p = 0 6 while p1 < m and p2 < n: 7 if nums1_copy[p1] < nums2[p2]: 8 nums1[p] = nums1_copy[p1] 9 p1 += 1 10 else: 11 nums1[p] = nums2[p2] 12 p2 += 1 13 p += 1 14 while p1 < m: 15 nums1[p] = nums1_copy[p1] 16 p1 += 1 17 p += 1 18 while p2 < n: 19 nums1[p] = nums2[p2] 20 p2 += 1 21 p += 1 </pre> (a) Base model	<pre> 1 def merge(self, nums1, m, nums2, n): 2 i = m - 1 3 j = n - 1 4 k = m + n - 1 5 while i >= 0 and j >= 0: 6 if nums1[i] > nums2[j]: 7 nums1[k] = nums1[i] 8 i -= 1 9 else: 10 nums1[k] = nums2[j] 11 j -= 1 12 k -= 1 13 while j >= 0: 14 nums1[k] = nums2[j] 15 j -= 1 16 k -= 1 </pre> (b) Fine-tuned model
--	---

Figure 6. **Example of generated code by base model and fine-tuned model.** The base model is DeepSeek Coder 6.7B. (a) Runtime: 0.6659 ms, BeyondX: 53.103. (b) Runtime: **0.4342 ms**, BeyondX: **82.750**.

5.6. Qualitative Results

Consider a coding problem where the task is to merge two sorted arrays into a single sorted array. Table 6 illustrates two examples: code (a) generated by the base model (DeepSeek Coder 6.7B) without preference optimization, and code (b) generated by the fine-tuned model with pref-

erence optimization. In this example, code (a) has a longer runtime due to operations that involve copying the arrays, whereas code (b) avoids these copying operations, making it approximately 1.5 times faster than code (a). Likewise, the BeyondX score of the fine-tuned model is about 1.5 times greater than that of the base model.

6. Discussion

Although BeyondX provides valuable insights into the efficiency of code generated by LLMs, it has several limitations that should be addressed in future work. One major limitation is its dependence on the system used for the evaluation, including variations in hardware, software, and execution environments. Relying on historical solutions and test cases also introduces bias, as these may not fully represent the diverse scenarios in real-world programming tasks. Another drawback is that BeyondX primarily focuses on runtime as a measure of efficiency, neglecting other important factors like memory usage and resource constraints, which are critical considerations in practical applications. To overcome these limitations, we will explore new metrics that reduce reliance on specific systems, historical solutions, and test cases. For example, incorporating time complexity could provide a more robust and system-independent measure of efficiency. Additionally, expanding the preference dataset to include more diverse information, such as time complexity, runtime, and resource constraints, would allow for a more comprehensive assessment of the generated code. By addressing these gaps, we can develop more reliable and holistic benchmarks to evaluate the efficiency of LLM-generated code.

7. Conclusion

In this work, we emphasize the importance of computational efficiency alongside functional correctness in evaluating the quality of LLM-generated code. To address the limitations of existing efficiency-aware benchmarks, we propose BeyondX, a CDF-based metric that overcomes the simplistic assumptions of metrics in prior approaches. By incorporating the full runtime distribution, BeyondX offers a more comprehensive and robust evaluation framework for assessing code efficiency, thereby improving the reliability of efficiency measurements. Furthermore, we design a preference dataset to train LLMs to produce code that balances both correctness and efficiency. Our experimental analysis demonstrates that the integration of DPO with IPO substantially enhances the efficiency of generated solutions. Collectively, these contributions advance both the evaluation and optimization of code-LLMs, providing a solid foundation for future research in efficient code generation with LLMs.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. arXiv preprint arXiv:2108.07732, 2021. [1](#), [2](#)
- [2] Mohammad Gheshlaghi Azar, Zhaohan Daniel Guo, Bilal Piot, Remi Munos, Mark Rowland, Michal Valko, and Daniele Calandriello. A general theoretical paradigm to understand learning from human preferences. In International Conference on Artificial Intelligence and Statistics, pages 4447–4455. PMLR, 2024. [1](#), [2](#), [4](#)
- [3] Michiel Bakker, Martin Chadwick, Hannah Sheahan, Michael Tessler, Lucy Campbell-Gillingham, Jan Bala-guer, Nat McAleese, Amelia Glaese, John Aslanides, Matt Botvinick, et al. Fine-tuning language models to find agreement among humans with diverse preferences. Advances in Neural Information Processing Systems, 35:38176–38189, 2022. [1](#), [3](#), [4](#)
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374, 2021. [1](#), [2](#)
- [5] Mingzhe Du, Anh Tuan Luu, Bin Ji, Qian Liu, and See-Kiong Ng. Mercury: A code efficiency benchmark for code large language models. In NeurIPS, 2024. [1](#), [2](#), [3](#)
- [6] Jiwoo Hong, Noah Lee, and James Thorne. Orpo: Monolithic preference optimization without reference model. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pages 11170–11189, 2024. [1](#), [2](#), [4](#)
- [7] Dong Huang, Jie M. Zhang, Yuhao Qing, and Heming Cui. Effibench: Benchmarking the efficiency of automatically generated code. In NeurIPS, 2024. [1](#), [2](#)
- [8] LeetCode. <https://leetcode.com/>. Accessed: December 17, 2024. [2](#), [3](#)
- [9] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. Advances in Neural Information Processing Systems, 36, 2024. [1](#), [2](#), [3](#), [4](#)
- [10] Haoran Xu, Amr Sharaf, Yunmo Chen, Weiting Tan, Lingfeng Shen, Benjamin Van Durme, Kenton Murray, and Young Jin Kim. Contrastive preference optimization: Pushing the boundaries of llm performance in machine translation. arXiv preprint arXiv:2401.08417, 2024. [1](#), [2](#), [4](#)